Lecture 05.15: Polymorphism — Method Overriding

📌 Executive Summary

Method overriding mechanics, runtime dynamic dispatch, virtual method tables, and overriding built-in methods.

🔬 Theoretical Foundations & Principles

1. Dynamic Method Dispatch

Polymorphism allows subclasses to provide specific implementations of methods defined in their superclass. At runtime, CPython dynamically inspects the instance's type and routes the call to the appropriate overridden implementation:

$$\text{Call}(o, m) \implies \text{lookup in } \text{type}(o).__\text{dict}__$$

2. Area Calculation Polymorphism

Given an abstract `Shape` interface, `Circle`, `Rectangle`, and `Triangle` each compute area using distinct formulas while presenting an identical `.area()` caller signature.

💻 Production Implementation & Code

```
import math

class Shape:
    def area(self) -> float:
        raise NotImplementedError("Subclasses must implement area()")

class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

    def area(self) -> float:
        return math.pi * (self.radius ** 2)

class Rectangle(Shape):
```

```
    def __init__(self, width: float, height: float):
        self.width = width
        self.height = height

    def area(self) -> float:
        return self.width * self.height

shapes = [Circle(3.0), Rectangle(4.0, 5.0)]
for s in shapes:
    print(f"Shape: {s.__class__.__name__:<10s} | Area: {s.area():.2f}")
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.